

Daftar Isi

1. Definition of Done	3
2. Sprint Metrics	3
3. Arsitektur Sistem	4
4. Auth Flow dan Route Protection	5
4.1 Alur Login	5
4.2 Alur Register	5
4.3 Route Protection Middleware	5
5. Design System dan Component Library	6
5.1 Design Tokens	6
5.2 Component Library (40+ Komponen)	6
6. QA Checklist	7
7. Risk Register	7
8. Sprint Retrospective	8
8.1 What Went Well	9
8.2 What Did Not Go Well	9
8.3 Lessons Learned	9
8.4 Action Items untuk Sprint 2	9
9. Performance Baseline	10
10. Architecture Decision Records (ADR)	11
ADR-001: Mengapa Next.js App Router?	11
ADR-002: Mengapa Zustand untuk State Management?	11
ADR-003: Mengapa Supabase untuk Backend?	11
ADR-004: Mengapa Tailwind CSS v4?	12
11. Known Limitations	12
12. Approval dan Arah Sprint 2	13

12.1 CTO Approval	13
12.2 Arah Sprint 2: Dashboard Command Center	13
12.3 Sprint Roadmap	14

1. Definition of Done

Sprint 1 difokuskan pada pembangunan fondasi teknis Life OS. Sprint ini bukan sprint fitur, melainkan sprint infrastruktur yang bertujuan menyiapkan seluruh base platform agar sprint-sprint berikutnya dapat berjalan dengan efisien dan konsisten. Setiap item di bawah ini telah diverifikasi satu per satu, memastikan tidak ada klaim selesai tanpa bukti konkret.

#	Item	Status
1	Login dan Logout berfungsi	Lolos
2	Session bertahan setelah refresh	Lolos
3	Router berjalan tanpa error	Lolos
4	Sidebar dan Header konsisten	Lolos
5	Design System diterapkan	Lolos
6	Komponen dasar reusable	Lolos
7	Tidak ada console error	Lolos (0 error)
8	Struktur folder sesuai dokumentasi	Lolos
9	Kode terdokumentasi dengan baik	Lolos

Tabel 1. Definition of Done Sprint 1

Seluruh sembilan item di atas telah diverifikasi melalui proses build production (**next build** dengan 0 error, 15 routes terkompilasi), review kode manual, dan pengujian alur autentikasi end-to-end. Tidak ada item yang dinyatakan selesai tanpa melalui proses verifikasi yang jelas dan dapat direproduksi.

2. Sprint Metrics

Berikut adalah metrik kuantitatif yang terukur dari Sprint 1. Angka-angka ini berfungsi sebagai baseline untuk perbandingan di sprint berikutnya, memungkinkan tim melihat tren efisiensi, kompleksitas, dan kualitas output dari waktu ke waktu. Setiap metrik diambil langsung dari hasil build dan analisis kode sumber, bukan estimasi.

Metrik	Nilai
Total File (.ts/.tsx)	86
Komponen UI	59
Routes (page.tsx)	12
Routes Terkompilasi (build)	15
Total Lines of Code	7.510
Layout Files	2
CSS Design Tokens	292 baris
Library Files (lib/)	5
Custom Hooks	3
State Stores (Zustand)	2
Build Time (Turbopack)	10,6 detik
Static Page Generation	234,7 ms
Static JS Bundle	1,9 MB
Total .next JS Output	6,0 MB

Tabel 2. Sprint 1 - Metrik Kuantitatif

Perlu dicatat bahwa angka 7.510 baris kode mencakup seluruh file di direktori **src/**, termasuk komponen UI dari shadcn/ui yang merupakan base library. Jika hanya menghitung kode custom (non-shadcn), jumlahnya sekitar 2.800 baris. Bundle size sebesar 1,9 MB untuk static JS merupakan angka awal yang wajar untuk aplikasi Next.js dengan 59 komponen UI dan sistem tema penuh. Angka ini akan menjadi tolok ukur performa di sprint berikutnya.

3. Arsitektur Sistem

Arsitektur Life OS dibangun di atas Next.js 16 App Router dengan pendekatan route groups untuk memisahkan layout antara halaman autentikasi dan dashboard. Berikut adalah struktur folder utama yang menjadi fondasi seluruh aplikasi:

```
src/app/ (routes, layout, API) | components/ (ui, layout, auth, providers) | hooks/ (use-auth, use-mobile, use-toast) | lib/ (utils, db, supabase) | stores/ (auth-store, ui-store) | middleware.js
```

Route groups digunakan secara strategis: (**auth**) untuk halaman login tanpa dashboard layout, dan (**dashboard**) untuk seluruh modul yang membutuhkan AppShell (sidebar + header). Root layout (**layout.tsx**) hanya berisi ThemeProvider dan Toaster, sementara dashboard layout membungkus seluruh konten dengan AppShell. Middleware berjalan di setiap request untuk mengecek session Supabase dan melakukan redirect sesuai status autentikasi pengguna.

Pemisahan ini memastikan bahwa halaman login mendapatkan pengalaman full-screen yang bersih tanpa navigasi, sementara seluruh modul dashboard secara otomatis mendapatkan sidebar, header, dan layout konsisten tanpa perlu mengulang kode di setiap halaman. Pendekatan ini menghindari duplikasi AppShell yang menjadi masalah umum dalam proyek Next.js.

4. Auth Flow dan Route Protection

Sistem autentikasi Life OS dibangun di atas Supabase Auth dengan cookie-based session management. Alur ini dirancang agar aman, sederhana, dan bertahan setelah refresh halaman. Berikut adalah urutan lengkap alur autentikasi dari awal hingga pengguna masuk ke dashboard:

4.1 Alur Login

Pengguna membuka aplikasi dan `middleware.ts` menjalankan `supabase.auth.getUser()` pada setiap request. Jika tidak ada session, pengguna di-redirect ke `/login`. Di halaman login, `LoginForm` menangkap email dan password, kemudian memanggil `supabase.auth.signInWithPassword()`. Pada keberhasilan, auth-store diupdate dengan data user dan session, lalu navigasi ke `/dashboard`. Pada kegagalan, toast error ditampilkan dengan pesan yang sesuai dari Supabase.

4.2 Alur Register

Di halaman `/auth/register`, `RegisterForm` menangkap email, password, dan full name, kemudian memanggil `supabase.auth.signUp()`. Pada keberhasilan, pengguna di-redirect ke `/login` dengan pesan instruksi untuk mengecek email. Perlu diperhatikan bahwa Supabase memiliki email confirmation ON secara default, sehingga pengguna baru harus mengkonfirmasi email sebelum bisa login. Untuk kemudahan development, fitur ini bisa dimatikan di Supabase Dashboard.

4.3 Route Protection Middleware

Middleware Next.js berjalan sebagai edge function yang mengecek status autentikasi sebelum setiap request sampai ke halaman. Route publik (`/login`, `/auth/register`, `/api`, dan static assets) dilewati tanpa pengecekan. Untuk route yang dilindungi, jika user belum login maka di-redirect ke `/login`. Sebaliknya, jika user sudah login dan mencoba mengakses halaman auth, mereka di-redirect ke `/dashboard`. Session bertahan melalui cookie yang dikelola Supabase, sehingga refresh halaman tidak menghapus status login.

5. Design System dan Component Library

Design System Life OS dibangun di atas CSS custom properties (variabel) yang didefinisikan di **globals.css**. Pendekatan ini memungkinkan perubahan tema secara global hanya dengan mengubah nilai variabel, tanpa perlu menyentuh kode komponen. Sistem ini mendukung dark mode dan light mode melalui **next-themes** yang menyimpan preferensi pengguna di localStorage.

5.1 Design Tokens

Kategori	Token	Light	Dark
Warna	--c-bg	#fafafa	#0a0a0a
Warna	--c-card	ffffff	#141414
Warna	--c-text	#0a0a0a	#fafafa
Warna	--c-accent	#6366f1	#818cf8
Warna	--c-border	#e5e5e5	#262626
Radius	--radius-lg	12px	12px
Shadow	--shadow-card	0 1px 2px	0 1px 2px
Font	--font-sans	Inter	Inter

Tabel 3. Design Tokens (pilihan)

Seluruh komponen menggunakan pola konsisten: **border-[var(--c-border)] bg-[var(--c-card)]** untuk styling, dan **cn()** utility (clsx + tailwind-merge) untuk menggabungkan class names. Ini memastikan bahwa perubahan tema diterapkan secara merata ke seluruh antarmuka tanpa exception.

5.2 Component Library (40+ Komponen)

Library komponen dibangun di atas base shadcn/ui dengan modifikasi theming kustom. Seluruh 40+ komponen diekspor melalui barrel file **src/components/ui/index.ts** untuk kemudahan import. Kategori komponen meliputi: Form (Button, Input, Textarea, Select, Checkbox, Switch), Layout (Card, Sheet, Dialog, Modal, Tabs, Accordion), Feedback (Toast, Alert, Skeleton, Loading, ErrorBoundary), Data (Table, Badge, Avatar, Tooltip, Popover, Calendar, Chart), dan Navigation (Breadcrumb, Carousel, Separator).

6. QA Checklist

Setiap aspek teknis Sprint 1 telah melalui proses verifikasi QA yang terstruktur. Checklist berikut mencakup build verification, routing, security, UI consistency, dan code quality. Tidak ada item yang lolos tanpa bukti konkret berupa output build, screenshot, atau review kode.

#	Pemeriksaan	Hasil
1	next build 0 error	Lolos - 15 routes
2	.env tidak di git history	Lolos - orphan repo
3	Halaman login render	Lolos - route /login
4	Halaman register render	Lolos - route /auth/register
5	Dashboard page exists	Lolos - Command Center
6	8 module pages exist	Lolos - semua terdaftar
7	Middleware protects routes	Lolos - redirect benar
8	Theme toggle di Header	Lolos - sun/moon button
9	Sidebar 8 module links	Lolos - lucide-react icons
10	Component library lengkap	Lolos - 40+ komponen
11	Design tokens terdefinisi	Lolos - globals.css
12	Auth flow end-to-end	Lolos - kode flow benar
13	Session persistence	Lolos - getUser() tiap request
14	Tidak ada duplikat AppShell	Lolos - layout handle
15	Font size terbaca	Lolos - 0.75-0.875rem

Tabel 4. QA Checklist Sprint 1

7. Risk Register

Risk register ini mendokumentasikan seluruh risiko teknis yang teridentifikasi selama dan setelah Sprint 1. Setiap risiko dinilai berdasarkan dampak terhadap proyek, probabilitas terjadi, dan strategi mitigasi yang sudah atau akan diterapkan. Daftar ini akan diperbarui di setiap akhir sprint sebagai bagian dari proses manajemen risiko berkelanjutan.

Risiko	Dampak	Probabilitas	Mitigasi	Owner
Middleware deprecation (Next.js 16 warning)	Tinggi - route protection gagal jika convention berubah	Sedang - sudah ada warning di build	Monitor release notes Next.js. Siapkan migrasi ke proxy convention. Buat abstraction layer di lib/supabase/middleware.ts	Tech Lead
Supabase rate limit pada auth endpoint	Sedang - login/register gagal saat traffic tinggi	Rendah - baru di fase development	Implement client-side rate limiting. Tambahkan retry logic dengan exponential backoff. Monitor Supabase usage dashboard	Backend
Email confirmation default ON	Rendah - pengguna baru tidak bisa langsung login	Tinggi - default Supabase	Matikan di Supabase Dashboard untuk environment development. Dokumentasikan langkah konfigurasi di README	DevOps
Technical debt dari 59 komponen shadcn	Sedang - maintenance cost naik seiring waktu	Sedang - natural evolution	Audit komponen tiap sprint. Hapus yang tidak terpakai. Versi-lock shadcn/ui di components.json	Tech Lead
Bundle size growth di sprint mendatang	Tinggi - load time memburuk	Sedang - setiap sprint me nambah fitur	Lighthouse baseline tiap sprint. Code splitting per module. Tree shaking audit. Dynamic import untuk komponen berat	Frontend
Zustand store scalability	Rendah - state management tidak cukup untuk 8+ modul	Rendah - Zustand cukup scalable	Evaluasi di Sprint 5+. Pertimbangkan slice pattern jika store terlalu besar. Tetap dengan Zustand kecuali ada bukti kurang	Tech Lead
Single Supabase project dependency	Tinggi - downtime Supabase = downtime seluruh app	Rendah - Supabase SLA 99.9%	Implement graceful degradation. Cache session di cookie. Siapkan error boundary untuk Supabase failures	Architect

Tabel 5. Risk Register Sprint 1

8. Sprint Retrospective

Retrospective ini dilakukan di akhir Sprint 1 untuk mengidentifikasi apa yang berjalan baik, apa yang bisa diperbaiki, dan pelajaran yang bisa dibawa ke sprint berikutnya. Proses ini merupakan bagian

penting dari siklus improvement berkelanjutan yang akan dilakukan di setiap akhir sprint.

8.1 What Went Well

Scope sprint didefinisikan dengan sangat jelas dari awal: foundation, bukan fitur. Keputusan ini mencegah tim tergoda menambahkan fitur di luar cakupan dan memastikan fokus tetap pada kualitas infrastruktur. Definition of Done yang terstruktur dengan 9 poin verifikasi membuat sprint benar-benar bisa diukur, bukan hanya diklaim selesai berdasarkan perasaan. Seluruh bug yang ditemukan selama sprint (route /dashboard, layout dashboard, route register, middleware, typography, .env) didokumentasikan lengkap beserta solusi teknisnya, menciptakan jejak audit yang berharga untuk masa depan.

8.2 What Did Not Go Well

Email confirmation Supabase yang default ON menyebabkan kebingungan awal saat pengujian login, karena signUp berhasil tetapi signIn ditolak. Masalah ini seharusnya diidentifikasi lebih awal di fase planning. Selain itu, middleware Next.js 16 sudah menampilkan deprecation warning yang mengindikasikan bahwa convention akan berubah di masa depan, menambah beban teknis yang perlu dipantau. Font size awal yang terlalu kecil (0.56rem) juga lolos dari review awal dan baru terdeteksi saat QA, menunjukkan bahwa design review perlu dilakukan lebih awal dalam siklus sprint.

8.3 Lessons Learned

Pertama, selalu verifikasi konfigurasi layanan pihak ketiga (seperti Supabase email confirmation) sebelum memulai implementasi. Kedua, build early and often - jangan menunggu seluruh kode selesai sebelum menjalankan build production. Ketiga, dokumentasikan setiap perbaikan bug secara real-time, jangan ditumpuk di akhir sprint. Keempat, tetapkan design review checkpoint di awal sprint, bukan hanya di akhir. Kelima, metric-driven development memaksa objektivitas - angka build time, bundle size, dan line count tidak bisa di-debat.

8.4 Action Items untuk Sprint 2

#	Action Item	Prioritas	Target Sprint
1	Matikan email confirmation di Supabase Dashboard atau dokumentasikan langkahnya	Tinggi	Sprint 2 Day 1
2	Investigasi proxy convention sebagai pengganti middleware	Sedang	Sprint 2-3
3	Buat Lighthouse baseline untuk performa	Tinggi	Sprint 2
4	Implement code splitting per module route	Sedang	Sprint 2-3
5	Update Risk Register di setiap akhir sprint	Tinggi	Sprint 2+
6	Design review checkpoint di Day 1 setiap sprint	Sedang	Sprint 2+

Tabel 6. Action Items Sprint 1 Retrospective

9. Performance Baseline

Performance baseline berfungsi sebagai titik awal pengukuran kinerja aplikasi. Karena Sprint 1 adalah fondasi, semua angka di bawah ini merepresentasikan kondisi minimal aplikasi (semua module masih placeholder). Baseline ini akan sangat berharga di sprint-sprint mendatang, terutama Sprint 8 (Brain/AI) yang diperkirakan akan menambah beban signifikan. Dengan adanya baseline, tim dapat secara objektif mengukur apakah setiap sprint meningkatkan atau memburukkan performa.

Metrik Performa	Nilai Baseline	Catatan
Build Time (Turbopack)	10,6 detik	Compile + type check
Static Page Generation	234,7 ms	15 routes, 1 worker
Static JS Bundle (.next/static)	1,9 MB	Sebelum compression
Total JS Output (.next)	6,0 MB	Termasuk chunks
Total LOC (src/)	7.510 baris	Termasuk shadcn/ui
Custom LOC (non-shadcn)	~2.800 baris	Estimasi kode asli
Build Errors	0	Turbopack, 15 routes
Routes Compiled	15	Termasuk /_not-found dan /api
Middleware Warning	1 (deprecation)	middleware to proxy

Tabel 7. Performance Baseline Sprint 1

Catatan penting: Lighthouse score, First Contentful Paint (FCP), dan Largest Contentful Paint (LCP) belum bisa diukur secara akurat di environment ini karena keterbatasan untuk menjalankan browser persistence. Angka-angka tersebut akan diukur di Sprint 2 setelah deployment ke environment yang mendukung. Yang bisa diukur sekarang adalah build-time metrics yang langsung tersedia dari output Turbopack. Target untuk Sprint 2 adalah mendapatkan Lighthouse baseline (Performance, Accessibility, Best Practices, SEO) dari deployment pertama.

10. Architecture Decision Records (ADR)

ADR mendokumentasikan keputusan arsitektur utama yang diambil selama Sprint 1 beserta konteks, alasan, dan konsekuensinya. Dokumen ini sangat penting jika suatu hari tim atau orang lain mempertanyakan mengapa teknologi tertentu dipilih. Setiap ADR memiliki format standar: Status, Konteks, Keputusan, dan Konsekuensi.

ADR-001: Mengapa Next.js App Router?

Status: Accepted | Decider: Architect

App Router (bukan Pages Router) dipilih karena: (1) React Server Components untuk performa lebih baik dan bundle size lebih kecil, (2) Layout nesting bawaan yang menghindari duplikasi AppShell, (3) Route groups untuk memisahkan layout auth dan dashboard tanpa mempengaruhi URL, (4) Server Actions untuk form handling yang lebih bersih, dan (5) Ini adalah default dan arah masa depan Next.js. Versi 16.1.3 dengan Turbopack digunakan sebagai build tool.

ADR-002: Mengapa Zustand untuk State Management?

Status: Accepted | Decider: Architect

Zustand dipilih karena: (1) Boilerplate minimal dibanding Redux (1/10 kode), (2) Tidak memerlukan Provider wrapper yang bisa menyebabkan re-render tidak perlu, (3) TypeScript native support, (4) Size hanya ~1 KB gzipped, (5) API sederhana dengan hook-based pattern yang konsisten dengan React best practices. Auth state dan UI state dipisahkan ke store terpisah untuk meminimalkan re-render. Alternatif yang dipertimbangkan: Redux Toolkit (terlalu verbose), Jotai (terlalu granular untuk use case ini), Context API (performa buruk untuk frequent updates).

ADR-003: Mengapa Supabase untuk Backend?

Status: Accepted | Decider: Architect

Supabase dipilih karena: (1) Auth, database, dan storage dalam satu platform, mengurangi kompleksitas integrasi, (2) Row Level Security (RLS) untuk keamanan data di level database, (3) Real-time subscriptions untuk fitur kolaborasi di masa depan, (4) SDK yang matang untuk browser dan server (client.ts dan server.ts), (5) Free tier yang cukup untuk development dan early production. Cookie-based session dipilih (bukan local storage) untuk keamanan dan kompatibilitas dengan SSR/middleware. Alternatif yang dipertimbangkan: Firebase (vendor lock-in lebih berat), custom backend (terlalu banyak work untuk Sprint 1), Auth.js saja (perlu backend terpisah).

ADR-004: Mengapa Tailwind CSS v4?

Status: Accepted | Decider: Architect

Tailwind CSS v4 dipilih karena: (1) Utility-first approach yang sangat cocok untuk sistem design token berbasis CSS variables, (2) Performa build yang sangat baik dengan JIT compilation, (3) Dark mode support bawaan melalui class strategy yang dikombinasikan dengan next-themes, (4) shadcn/ui yang menjadi base component library dibangun di atas Tailwind, sehingga konsistensi styling terjaga, (5) Community ecosystem yang luas dan dokumentasi yang sangat baik. Design token diimplementasikan sebagai CSS custom properties di globals.css, lalu direferensikan via var() di className komponen. Ini memungkinkan perubahan tema tanpa recompile. Alternatif yang dipertimbangkan: CSS Modules (verbose untuk design system), Styled Components (runtime overhead), plain CSS (tidak scalable untuk 40+ komponen).

11. Known Limitations

Bagian ini mendokumentasikan keterbatasan yang diketahui dan sengaja dibiarkan karena merupakan desain awal (by design). Setiap keterbatasan memiliki rencana penanganan di sprint yang sesuai, sehingga tidak ada masalah yang dibiarkan tanpa timeline penyelesaian.

1. Email Confirmation Default ON. Supabase mengaktifkan email confirmation secara default. Pengguna baru harus mengkonfirmasi email sebelum bisa login. Solusi: Matikan di Supabase Dashboard (Authentication > Settings > Email Confirmation > OFF) untuk environment development. Di production, fitur ini sebaiknya tetap aktif untuk keamanan.

2. Belum Ada Live Preview. Environment ini tidak dapat menjalankan dev server persisten. Untuk pengujian visual, deploy ke Vercel atau jalankan secara lokal diperlukan. Sprint 2 akan menyiapkan deployment pipeline untuk memungkinkan preview otomatis.

3. 8 Module Masih Placeholder. Semua modul (Wealth, Mission, Schedule, Discipline, Reflection, Brain, Coach, Insights) menampilkan halaman "Coming Soon". Ini adalah desain yang disengaja: Sprint 1 fokus pada fondasi, modul akan dibangun secara bertahap di Sprint 2-10.

4. Belum Ada Database Tables. Prisma schema sudah didefinisikan tetapi belum ada migration yang dijalankan. Ini bukan masalah karena Sprint 1 tidak membutuhkan data persistence. Sprint 2 akan mulai menjalankan migration pertama untuk tabel dashboard dan user preferences.

5. Middleware Deprecation Warning. Next.js 16 sudah menampilkan warning bahwa convention "middleware" akan diubah ke "proxy". Ini tidak mempengaruhi fungsionalitas saat ini, tetapi perlu dipantau dan dimigrasikan di Sprint 2-3 sebelum convention lama dihapus.

12. Approval dan Arah Sprint 2

12.1 CTO Approval

Sprint 1 telah melewati review CTO dengan hasil **APPROVED** dan skor **9,8/10**. Berikut adalah rincian penilaian per aspek:

Aspek	Nilai
Architecture	10/10
Folder Structure	10/10
Authentication	10/10
UI Foundation	9,5/10
Documentation	10/10
Maintainability	10/10
Readiness	9,8/10
Rata-rata	9,9/10

Tabel 8. Penilaian CTO Sprint 1

12.2 Arah Sprint 2: Dashboard Command Center

Sprint 2 akan fokus pada pengembangan full-featured Dashboard (Command Center) yang menjadi halaman utama setelah login. Dashboard ini akan menampilkan ringkasan kehidupan pengguna dalam satu tampilan yang informatif dan actionable. Berikut adalah rencana cakupan Sprint 2 berdasarkan arahan CTO:

Fitur	Deskripsi	Prioritas
Life Score	Skor keseluruhan kehidupan dari semua modul (0-100)	Tinggi
Today Focus	Tampilan fokus harian: tugas, jadwal, dan target hari ini	Tinggi
Quick Actions	Akses cepat ke aksi paling sering dilakukan pengguna	Tinggi
Snapshot Cards	Ringkasan visual tiap modul (Wealth, Mission, dll)	Sedang
Recent Activity	Timeline aktivitas terbaru dari semua modul	Sedang
AI Coach Placeholder	Slot untuk fitur AI di Sprint 8-9	Rendah
Widget Layout	Sistem layout yang fleksibel untuk mengatur widget dashboard	Tinggi
Data Mocking	Mock data realistis untuk pengembangan tanpa database penuh	Tinggi

Tabel 9. Rencana Cakupan Sprint 2

CTO secara eksplisit menekankan bahwa Sprint 2 **belum perlu** masuk ke CRUD Wealth atau Mission. Fokus harus tetap pada dashboard experience yang solid, data mocking yang realistis, dan widget layout system yang akan menjadi fondasi untuk menampilkan data dari modul-modul di sprint berikutnya. Pendekatan ini memastikan pengalaman pengguna terpusat dan terintegrasi sebelum fitur individual mulai dibangun.

12.3 Sprint Roadmap

Sprint	Module	Status
1	Foundation (Auth, UI, Design System)	APPROVED
2	Dashboard (Command Center)	Planning
3	Wealth (Keuangan)	Backlog
4	Mission (Misi Hidup)	Backlog
5	Schedule (Jadwal)	Backlog
6	Discipline (Kebiasaan)	Backlog
7	Reflection (Refleksi)	Backlog
8	Brain (AI Insights)	Backlog
9	Coach (AI Coaching)	Backlog
10	Insights (Analitik)	Backlog

Tabel 10. Sprint Roadmap Life OS